

Correction des Travaux Dirigés MERISE

Corrigé détaillé des 26 exercices
avec remarques pédagogiques

Année universitaire 2026 – 2027

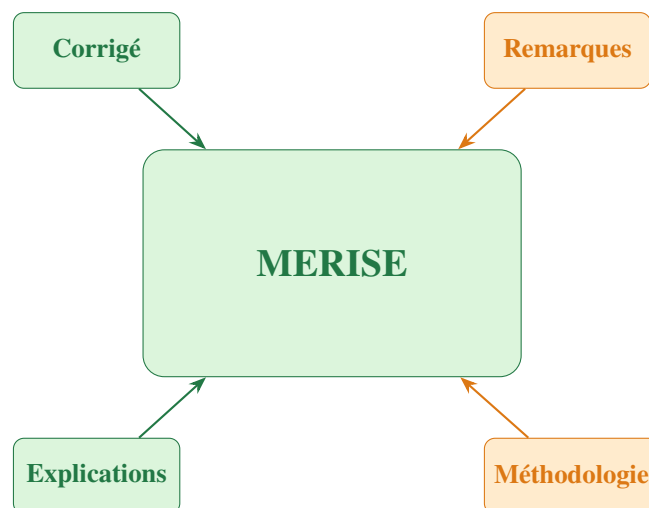


Table des matières

1	Chapitre 1 : Introduction à MERISE	2
2	Chapitre 2 : Modélisation conceptuelle des données (MCD)	6
3	Chapitre 3 : Modélisation logique des données (MLD)	13
4	Chapitre 4 : Modélisation physique des données (MPD)	19

1 Chapitre 1 : Introduction à MERISE

Correction — Exercice 1.1 : QCM — Présentation générale de MERISE

1. **Réponses b) et d)** — MERISE est à la fois une **méthode de conception de systèmes d'information** (approche globale) et une **approche de modélisation de données** (technique de représentation). Ce n'est ni un langage de programmation ni un SGBD.
2. **Réponse b)** — MERISE a été créée dans les **années 1970**, à l'initiative du ministère français de l'Industrie. Les travaux ont été menés par la SEMA (René Colletti, Hubert Tardieu) à partir de 1976-1977.
3. **Réponse a)** — Les trois niveaux d'abstraction de MERISE sont le **conceptuel**, le **logique** et le **physique**. Ces trois niveaux forment le cycle d'abstraction, principe fondamental de la méthode.
4. **Réponse a)** — Le niveau conceptuel correspond à la **vue métier**, indépendante de toute considération technique ou organisationnelle. Il exprime le « quoi » (que fait le système ?) sans se préoccuper du « comment ».

Remarque

MERISE est une méthode française de conception de SI, créée à la fin des années 1970. Son originalité réside dans la séparation entre les vues **données** et **traitements** à chaque niveau d'abstraction, ce qui permet une approche systématique et rigoureuse de la conception.

Correction — Exercice 1.2 : Les niveaux d'abstraction

Tableau complété :

Niveau	Description	Modèles
Conceptuel	Vue métier, indépendante de toute technique. Exprime le « quoi ».	MCD, MCT
Logique	Traduction du conceptuel en tenant compte de l'organisation.	MLD, MLT
Physique	Implémentation technique dans un SGBD particulier.	MPD, MOT

Classification des affirmations :

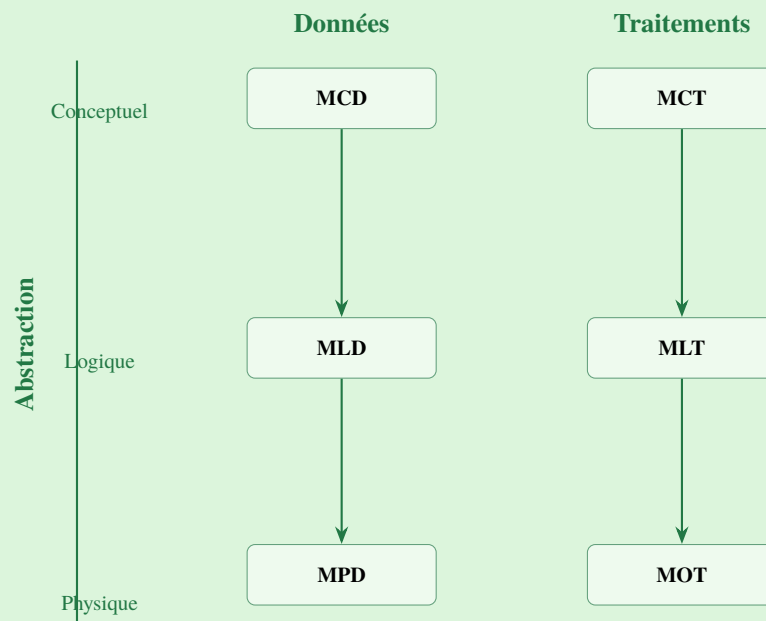
1. **C** — « Un client peut passer plusieurs commandes » → exprime une règle métier (niveau conceptuel).
2. **P** — « La table CLIENT possède un index sur le champ NOM » → concerne l'implémentation physique.
3. **C** — « L'association LIVRER relie les entités FOURNISSEUR et PRODUIT » → modélisation conceptuelle.
4. **P** — « Le champ `date_commande` est de type DATE » → choix de type physique.
5. **L** — « La relation COMMANDE contient la clé étrangère `ref_client` » → structure logique (relation, clé étrangère).

Astuce de classification

Posez-vous la question : « Cette affirmation dépend-elle d'un choix technique ? » Si oui, c'est le niveau physique. Si elle décrit une règle métier pure, c'est le conceptuel. Si elle décrit une structure logique sans choix de SGBD, c'est le logique.

Correction — Exercice 1.3 : Le cycle d'abstraction de MERISE

1. **Différence entre conceptuel et logique** : Le niveau **conceptuel** décrit le système de manière abstraite, en se concentrant sur le « quoi » : quelles sont les entités du métier, leurs relations, les règles de gestion ? Il est totalement indépendant de toute technologie.
Le niveau **logique** est une traduction du conceptuel en tenant compte de l'organisation et des choix d'architecture. On y décrit les structures de données (relations, tables) et les traitements sous une forme plus proche de l'implémentation, sans encore dépendre d'un SGBD particulier.
2. **Pourquoi séparer données et traitements ?** MERISE sépare les deux vues car elles évoluent de manière relativement indépendante. Les données (structure du SI) changent moins fréquemment que les traitements (processus métier). Cette séparation permet de modifier les traitements sans remettre en cause la structure des données, et inversement.
3. **Schéma du cycle d'abstraction** :



4. **Exemple concret** : Considérons la gestion des commandes :
 - **Conceptuel** : Entité CLIENT, entité COMMANDE, association « Passer » avec cardinalités (1,n)/(1,n).
 - **Logique** : Relation COMMANDE(num_commande, date, #num_client) — la clé étrangère apparaît.
 - **Physique** : Table SQL COMMANDE avec INTEGER, DATE, FOREIGN KEY et un index sur num_client.

Remarque

Le passage d'un niveau à l'autre se fait par des **règles de transformation** bien définies. Le conceptuel est le plus stable ; le physique est le plus dépendant des choix techniques. On peut changer de SGBD (physique) sans modifier le MCD (conceptuel).

Correction — Exercice 1.4 : Vrai ou Faux

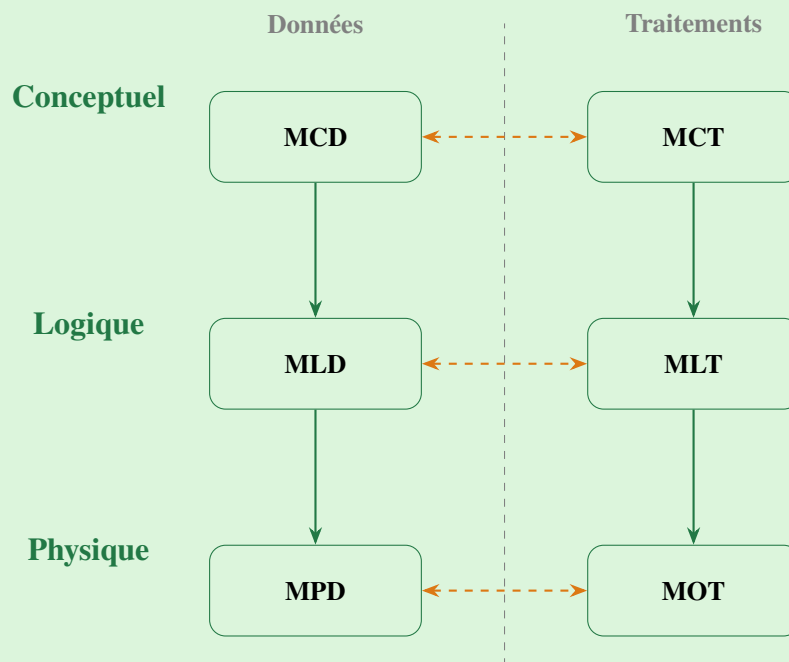
1. **Faux** — MERISE n'est pas réservée aux bases de données relationnelles. C'est une méthode générale de conception de SI qui peut être appliquée à différents types de SGBD (relationnel, objet, NoSQL...). Le MCD est indépendant du modèle logique choisi.
2. **Vrai** — Le MCD (Modèle Conceptuel des Données) est indépendant de tout SGBD. Il décrit la réalité du métier sans se préoccuper de la manière dont les données seront stockées.
3. **Vrai** — Le niveau physique dépend du SGBD choisi. C'est au niveau physique que l'on choisit les types de données, les index, les stratégies de stockage, etc.
4. **Faux** — Le MLD et le MPD ne sont pas la même chose. Le MLD est une description logique des données (relations, clés) indépendante du SGBD. Le MPD est la traduction du MLD dans le langage spécifique d'un SGBD (types SQL, index, etc.).
5. **Faux** — MERISE traite à la fois les **données** et les **traitements**. À chaque niveau d'abstraction, il existe une vue données et une vue traitements : MCD/MCT, MLD/MLT, MPD/MOT.
6. **Vrai** — Le passage du MCD au MLD se fait par des règles de transformation automatiques (entité → relation, association 1 : n → clé étrangère, association n : m → relation supplémentaire, etc.).

Points clés

La distinction MLD/MPD est souvent source de confusion. Retenez : le MLD est **logique** (indépendant du SGBD), le MPD est **physique** (dépend du SGBD). Le MLD décrit des **relations** ; le MPD décrit des **tables SQL**.

Correction — Exercice 1.5 : Les courbes MERISE

1. et 2. Schéma des courbes de MERISE :



3. **Pourquoi « cycle d'abstraction » ?** Ce schéma est appelé cycle d'abstraction car il représente les différents niveaux de détail auxquels on peut décrire un système d'information. Du plus abstrait (conceptuel) au plus concret (physique), chaque niveau affine et précise le précédent. Le terme « cycle » souligne que la conception est itérative : on peut remonter d'un niveau à l'autre pour affiner le modèle.

Remarque

Les courbes de MERISE illustrent la **séparation des préoccupations** : à chaque niveau, on traite séparément les données et les traitements. Les flèches horizontales montrent que données et traitements interagissent mais restent modélisés de manière indépendante.

2 Chapitre 2 : Modélisation conceptuelle des données (MCD)

Correction — Exercice 2.1 : QCM — Entités et associations

1. **Réponse a)** — Une entité dans un MCD représente un **objet du monde réel ayant une existence propre**. Ce n'est pas une table (niveau physique), ni un événement (c'est le rôle de l'association), ni un attribut.
2. **Réponses b) et c)** — Une association peut être **réflexive** (relier une entité à elle-même) ou **ternaire** (relier plus de deux entités). Elle ne relie pas toujours exactement deux entités, et elle n'a pas obligatoirement d'attributs.
3. **Réponses a) et c)** — L'identifiant d'une entité est un attribut à **valeur unique** pour chaque occurrence. Il peut être **composé** de plusieurs propriétés (identifiant composite), mais il n'est pas obligatoirement numérique et n'est pas calculé.
4. **Réponse a)** — Les cardinalités indiquent le **nombre minimum et maximum** d'occurrences d'une entité pouvant participer à une association.

Remarque

Retenez la règle des cardinalités : on note toujours la cardinalité **minimum** puis la cardinalité **maximum**, séparées par une virgule. Exemple : (1,n) signifie « au moins 1, au plus n ».

Correction — Exercice 2.2 : Bibliothèque — Entités, associations et attributs

1. **Entités et attributs :**

Entité	Attributs
LIVRE	ISBN, titre, auteur, année_publication
ABONNÉ	num_carte, nom, prénom, adresse
2. **Association et cardinalités :**
EMPRUNTER entre ABONNÉ et LIVRE :
 - ABONNÉ –(0,n)– EMPRUNTER : un abonné peut emprunter 0 ou plusieurs livres.
 - LIVRE –(0,1)– EMPRUNTER : un livre est emprunté par 0 ou 1 abonné à la fois.
3. **Attributs de l'association :** Oui, **date_emprunt** et **date_retour_prévue** dépendent de l'association EMPRUNTER. Ils ne peuvent pas être rattachés ni à LIVRE seul ni à ABONNÉ seul.



Remarque

Un attribut qui dépend à la fois de deux entités (comme **date_emprunt** dépend de l'abonné ET du livre) est un **attribut d'association**. C'est un indice que l'association porte des informations propres.

Correction — Exercice 2.3 : Cardinalités**1. Employé – Département :**

- Employé → Département : (1,1) — chaque employé travaille dans un seul département.
- Département → Employé : (1,n) — un département emploie au moins un employé.

2. Professeur – Matière :

- Professeur → Matière : (1,n) — un professeur enseigne au moins une matière.
- Matière → Professeur : (1,n) — une matière est enseignée par au moins un professeur.

3. Client – Commande :

- Client → Commande : (1,n) — un client passe au moins une commande.
- Commande → Client : (1,1) — une commande est passée par un seul client.

4. Produit – Fournisseur :

- Produit → Fournisseur : (0,n) — un produit peut être fourni par 0 ou n fournisseurs.
- Fournisseur → Produit : (1,n) — un fournisseur fournit au moins un produit.

5. Personne – Téléphone :

- Personne → Téléphone : (1,n) — une personne possède au moins un numéro.
- Téléphone → Personne : (1,1) — un numéro appartient à une seule personne.

Méthode pour déterminer les cardinalités

Pour chaque entité, posez-vous deux questions : 1) « Une occurrence de cette entité peut-elle ne pas participer à l'association ? » (cardinalité min); 2) « Combien d'occurrences de l'autre entité une occurrence peut-elle être liée ? » (cardinalité max). Le minimum est toujours 0 ou 1 ; le maximum est 1 ou n.

Correction — Exercice 2.4 : MCD — Cabinet médical

1. Entités et attributs :

Entité	Attributs
MÉDECIN	<u>num_ordre</u> , nom, spécialité, tarif
PATIENT	<u>num_sécu</u> , nom, prénom, date_naissance
ORDONNANCE	<u>num_oronnance</u> , date
MÉDICAMENT	<u>code_médicament</u> , nom_médicament

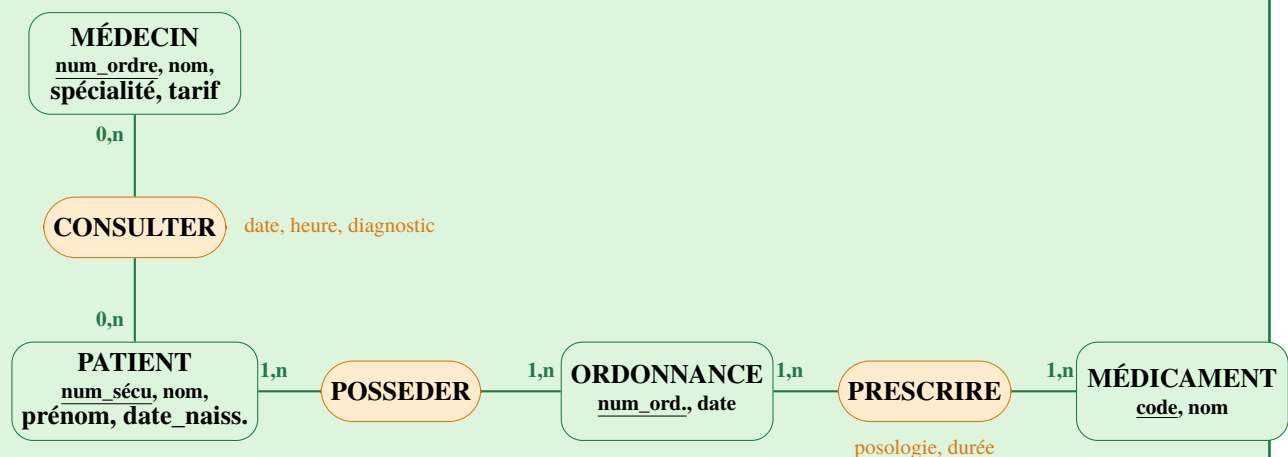
2. Associations et cardinalités :

Association	Entité 1	Entité 2	Cardinalités
CONSULTER	MÉDECIN	PATIENT	(0,n) – (0,n)
POSSEDER	PATIENT	ORDONNANCE	(1,n) – (1,n)
PRESCRIRE	ORDONNANCE	MÉDICAMENT	(1,n) – (1,n)

Attributs d'associations :

- CONSULTER : date, heure, diagnostic
- PRESCRIRE : posologie, durée

3. MCD complet :



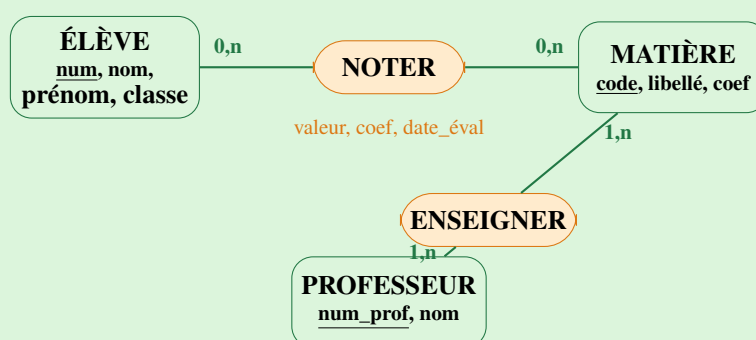
Remarque

L'association CONSULTER est une association n : m car un médecin peut voir plusieurs patients et un patient peut consulter plusieurs médecins. Elle possède des attributs (date, heure, diagnostic) car ces informations dépendent à la fois du médecin et du patient.

Correction — Exercice 2.5 : MCD — Résultats scolaires**1. Entités, associations et cardinalités :**

Entité	Attributs
ÉLÈVE	<u>num_élève</u> , nom, prénom, classe
MATIÈRE	<u>code</u> , libellé, coefficient
PROFESSEUR	<u>num_prof</u> , nom, prénom

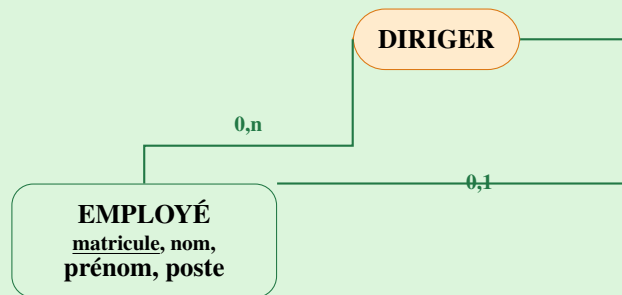
Association	Entités	Cardinalités	Attributs
NOTER	ÉLÈVE — MATIÈRE	(0,n) – (0,n)	valeur, coef, date_éval
ENSEIGNER	PROFESSEUR — MATIÈRE	(1,n) – (1,n)	—
ENSEIGNER_CLASSE	PROFESSEUR — CLASSE	(0,n) – (1,n)	—

2. MCD complet :

3. **Associations ternaires ?** Non, il n'y a pas d'association ternaire dans ce MCD. L'association NOTER est binaire (entre ÉLÈVE et MATIÈRE). Le professeur qui enseigne la matière est relié par l'association ENSEIGNER, séparée de la note.

Remarque

Une association ternaire relie trois entités simultanément. Ici, la note dépend de l'élève et de la matière, pas du professeur. Le professeur est relié à la matière via ENSEIGNER, ce qui est une association binaire distincte.

Correction — Exercice 2.6 : Association réflexive**1. MCD avec association réflexive :****2. Cardinalités :**

- Côté « dirige » (supérieur) : (0,n) — un employé peut diriger 0 ou n subordonnés.
- Côté « est dirigé par » (subordonné) : (0,1) — un employé a au plus un supérieur direct.

3. Différence entre les deux rôles : L'association réflexive DIRIGER relie EMPLOYÉ à EMPLOYÉ, mais les deux « pattes » jouent des rôles différents :

- Le rôle « dirige » (cardinalité 0,n) : l'employé est le chef, il peut avoir plusieurs subordonnés.
- Le rôle « est dirigé par » (cardinalité 0,1) : l'employé est le subordonné, il a au plus un chef.

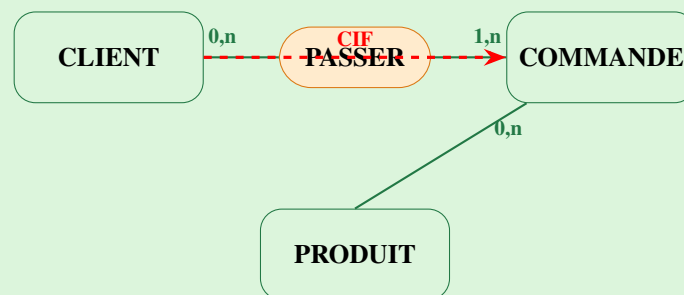
Le directeur général a la cardinalité (0,n) du côté « dirige » mais (0,1) du côté « est dirigé » avec min=0 (il n'a pas de supérieur).

Remarque

Les associations réflexives sont fréquentes pour modéliser les hiérarchies (supérieur/subordonné), les parentés (parent/enfant), ou les précédences (tâche antérieure/tâche postérieure). La cardinalité 0,1 du côté subordonné traduit le fait qu'un employé a au plus un supérieur direct.

Correction — Exercice 2.7 : CIF et contraintes d'intégrité

1. **Définition d'une CIF** : Une Contrainte d'Intégrité Fonctionnelle (CIF) est une dépendance fonctionnelle entre entités dans un MCD. Elle exprime qu'une entité est **fonctionnellement déterminée** par une (ou plusieurs) autre(s) entité(s) dans le contexte d'une association. En pratique, une CIF indique qu'une des entités participantes « dépend » d'une autre pour son identification dans l'association.
2. **CIF dans l'exercice 2.4 (cabinet médical)** : Dans le MCD du cabinet médical : l'ordonnance est prescrite par un médecin pour un patient lors d'une consultation. L'association POSSEDER entre PATIENT et ORDONNANCE a une cardinalité (1,n)/(1,n), mais on peut identifier une CIF : une ordonnance est **fonctionnellement dépendante** d'un patient (un patient possède une ordonnance, mais une ordonnance appartient à un seul patient). Cela signifie qu'on peut réécrire le MCD en insérant la clé de PATIENT dans ORDONNANCE.
3. **CIF pour l'association ternaire Passer** : On sait qu'une commande est obligatoirement passée par un seul client. Donc : COMMANDE → CLIENT est une CIF.



Remarque

La CIF se traduit au niveau logique par l'insertion de la clé primaire de l'entité source (CLIENT) comme clé étrangère dans l'entité cible (COMMANDE). Une CIF simplifie le MLD en évitant la création d'une table supplémentaire pour l'association.

Correction — Exercice 2.8 : Dépendances fonctionnelles

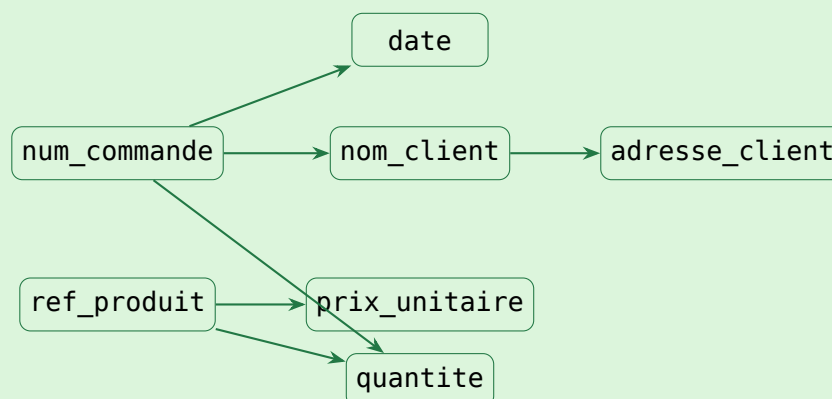
Soit la relation `COMMANDE(num_commande, date, nom_client, adresse_client, ref_produit, quantite, prix_unitaire)`.

1. **Dépendances fonctionnelles élémentaires :**

- `num_commande → date`
- `num_commande → nom_client`
- `num_commande → adresse_client`
- `(num_commande, ref_produit) → quantite`
- `ref_produit → prix_unitaire`

2. **DF directes non élémentaires :** `num_commande → nom_client → adresse_client` est une DF transitive (`num_commande` détermine `nom_client`, et `nom_client` détermine potentiellement `adresse_client`). La relation n'est pas en 3NF.

3. **Graphe des dépendances fonctionnelles :**



4. **MCD correspondant :** La relation `COMMANDE` n'est pas en forme normale. Le MCD comporte :

- Entité `COMMANDE` : `num_commande`, `date`
- Entité `CLIENT` : `nom_client`, `adresse_client`
- Entité `PRODUIT` : `ref_produit`, `prix_unitaire`
- Association `COMPORTER` (`COMMANDE` — `PRODUIT`) : `quantite`, cardinalités (1,n)/(1,n)
- Association `PASSER` (`CLIENT` — `COMMANDE`) : cardinalités (1,n)/(1,n)

Remarque

L'analyse des dépendances fonctionnelles permet de **normaliser** la relation et de remonter au MCD. La relation initiale viole la 3NF car `adresse_client` dépend transitivement de `num_commande` via `nom_client`.

3 Chapitre 3 : Modélisation logique des données (MLD)

Correction — Exercice 3.1 : QCM — Passage du MCD au MLD

1. **1. Réponse a)** — Lors du passage du MCD au MLD, une entité se transforme en **relation** (ou table dans le vocabulaire relationnel). Chaque propriété de l'entité devient un attribut de la relation.
2. **2. Réponse b)** — Une association de type 1 :n se transforme par l'ajout de la **clé primaire de la table « 1 »** comme **clé étrangère dans la table « n »**. Aucune table supplémentaire n'est créée.
3. **3. Réponse b)** — Une association de type n :m se transforme par la **création d'une nouvelle table** contenant les clés primaires des deux entités comme clé primaire composée.
4. **4. Réponse b)** — La clé primaire d'une table issue d'une association n :m est la **concaténation des clés primaires** des entités associées.

Règles de transformation MCD → MLD

Élément MCD	Transformation MLD
Entité	Relation ayant même nom et mêmes attributs, plus la clé primaire
Association 1 :1	Fusion des deux relations ou ajout d'une clé étrangère
Association 1 :n	Ajout de la clé primaire de « 1 » en clé étrangère dans « n »
Association n :m	Création d'une nouvelle relation avec clé composée
Association réflexive	Duplication de la clé + clé étrangère dans la même relation
Association ternaire	Nouvelle relation avec clé composée des 3 (ou plus) clés

Correction — Exercice 3.2 : MLD — Bibliothèque

En reprenant le MCD de l'exercice 2.2 :

1. Transformation MCD → MLD :

Relation	Attributs	Clé primaire	Clé étrangère
LIVRE	ISBN, titre, auteur, année_pub	<u>ISBN</u>	—
ABONNÉ	num_carte, nom, prénom, adresse	<u>num_carte</u>	—
EMPRUNTER	#ISBN, #num_carte, date_emprunt, date_retour	<u>#ISBN, #num_carte</u>	#ISBN, #num_carte

Explication :

- LIVRE et ABONNÉ sont des entités → relations directes.
- EMPRUNTER est une association n : m → nouvelle relation. La cardinalité (0,1) du côté LIVRE signifie qu'un livre est emprunté par 0 ou 1 abonné à la fois, mais l'association reste n : m car un abonné peut emprunter plusieurs livres.
- Les attributs `date_emprunt` et `date_retour_prévue` de l'association deviennent des attributs de la relation EMPRUNTER.

2. Vérification des règles :

- Entité → Relation : (LIVRE, ABONNÉ)
- Association n : m → Nouvelle relation : (EMPRUNTER)
- Attributs d'association → Attributs de la relation : (date_emprunt, date_retour)

3. Contraintes d'intégrité référentielle :

- #ISBN dans EMPRUNTER doit exister dans LIVRE.ISBN
- #num_carte dans EMPRUNTER doit exister dans ABONNÉ.num_carte

Remarque

Attention : la cardinalité (0,1) du côté LIVRE signifie qu'un livre ne peut être emprunté que par au plus un abonné à un instant donné. Cela n'empêche pas l'association d'être traitée comme n : m au niveau MLD, car l'historique des emprunts peut nécessiter la table EMPRUNTER.

Correction — Exercice 3.3 : MLD — Cabinet médical

En reprenant le MCD de l'exercice 2.4 :

1. **Transformation MCD → MLD :**

Relation	Attributs	Clé primaire
MÉDECIN	num_ordre, nom, spécialité, tarif	<u>num_ordre</u>
PATIENT	num_sécu, nom, prénom, date_naissance	<u>num_sécu</u>
CONSULTER	#num_ordre, #num_sécu, date, heure, diagnostic	<u>#num_ordre, #num_sécu</u>
ORDONNANCE	num_ordonnance, date, #num_sécu	<u>num_ordonnance</u>
PRESCRIRE	#num_ordonnance, #code_médicament, posologie, durée	<u>#num_ordonnance, #code_médicament</u>
MÉDICAMENT	code_médicament, nom_médicament	<u>code_médicament</u>

2. **Clés primaires et étrangères :** Voir tableau ci-dessus.

3. **Contraintes d'intégrité :**

- #num_ordre dans CONSULTER → référence MÉDECIN.num_ordre
- #num_sécu dans CONSULTER → référence PATIENT.num_sécu
- #num_sécu dans ORDONNANCE → référence PATIENT.num_sécu
- #num_ordonnance dans PRESCRIRE → référence ORDONNANCE.num_ordonnance
- #code_médicament dans PRESCRIRE → référence MÉDICAMENT.code_médicament

Remarque

L'association POSSEDER (1,n)/(1,n) entre PATIENT et ORDONNANCE est de type 1 :n car une ordonnance appartient à un seul patient. La clé de PATIENT est donc insérée comme clé étrangère dans ORDONNANCE. L'association CONSULTER (0,n)/(0,n) est n :m et donne lieu à une relation supplémentaire.

Correction — Exercice 3.4 : Transformation d'associations particulières

1. **Association réflexive Superviser (0,n / 0,1) :**
EMPLOYÉ(matricule, nom, prénom, poste, #matricule_superviseur)
La clé étrangère #matricule_superviseur référence le matricule du supérieur hiérarchique dans la même table. La cardinalité 0,1 signifie que la clé étrangère peut être NULL (pas de supérieur).
2. **Association ternaire Approvisionner (0,n / 0,n / 0,n) :**
FOURNISSEUR(num_fournisseur, nom)
ENTREPÔT(num_entrepôt, lieu)
PRODUIT(ref_produit, libellé)
APPROVISIONNER(#num_fournisseur, #num_entrepôt, #ref_produit, date, quantite)
Clé primaire : #num_fournisseur, #num_entrepôt, #ref_produit
3. **Association 1 :1 Gérer (1,1 / 0,1) :**
Deux solutions possibles :
 - Fusion des deux relations : SERVICE(num_service, nom, #matricule_responsable) où le matricule du responsable est unique (UNIQUE).
 - Ou ajout de la clé étrangère dans EMPLOYÉ : EMPLOYÉ(matricule, ..., #num_service_géré) avec contrainte UNIQUE.
4. **Association n :m Noter avec attributs :**
ÉLÈVE(num_élève, nom, prénom, classe)
MATIÈRE(code, libellé, coefficient)
NOTER(#num_élève, #code, note, date)
Clé primaire : #num_élève, #code

Remarque

Pour les associations 1 :1, il est souvent préférable de fusionner les deux relations ou d'ajouter une clé étrangère avec contrainte UNIQUE. Pour les associations réflexives, la clé étrangère référence la clé primaire de la même table.

Correction — Exercice 3.5 : Vérification de MLD

Soit le MLD : COMMANDE(num_commande, date, #ref_client),
 LIGNE_COMMANDE(#num_commande, #ref_produit, quantite),
 PRODUIT(ref_produit, designation, prix_unitaire), CLIENT(num_client,
 nom, adresse).

1. Clés primaires et étrangères :

Relation	Clé primaire	Clé étrangère
COMMANDE	<u>num_commande</u>	#ref_client → CLIENT
LIGNE_COMMANDE	<u>#num_commande, #ref_produit</u>	#num_commande → COMMANDE, #ref_produit →
PRODUIT	<u>ref_produit</u>	—
CLIENT	<u>num_client</u>	—

2. MCD correspondant :

- Entités : CLIENT, COMMANDE, PRODUIT
- Association PASSER (CLIENT — COMMANDE) : cardinalités (1,n) / (1,1) car #ref_client est dans COMMANDE (1 :n)
- Association COMPORTER (COMMANDE — PRODUIT) : cardinalités (1,n) / (1,n) car LIGNE_COMMANDE est une table intermédiaire (n :m) avec attribut **quantite**

3. Conformité :

- Entités → Relations :
- Association 1 :n (PASSER) → Clé étrangère dans COMMANDE :
- Association n :m (COMPORTER) → Table LIGNE_COMMANDE :
- Attribut d'association (quantite) dans LIGNE_COMMANDE :

4. Contrainte : un produit une seule fois par commande. La clé primaire de LIGNE_COMMANDE est déjà (#num_commande, #ref_produit), ce qui garantit qu'un même produit ne peut pas apparaître deux fois dans la même commande. La contrainte est **déjà respectée** par la clé primaire composée.**Remarque**

La clé primaire composée d'une table issue d'une association n :m garantit naturellement l'unicité du couple (entité1, entité2). C'est un avantage de la modélisation relationnelle.

Correction — Exercice 3.6 : MLD — Résultats scolaires

En reprenant le MCD de l'exercice 2.5 :

1. **Transformation MCD → MLD :**

Relation	Attributs	Clé primaire	Clé étrangère
ÉLÈVE	num_élève, nom, prénom, classe	<u>num_élève</u>	—
MATIÈRE	code, libellé, coefficient	<u>code</u>	—
PROFESSEUR	num_prof, nom, prénom	<u>num_prof</u>	—
NOTER	#num_élève, #code, valeur, coef, date_éval	<u>#num_élève, #code</u>	#num_élève, #code
ENSEIGNER	#num_prof, #code	<u>#num_prof, #code</u>	#num_prof, #code

2. Les clés primaires sont soulignées, les clés étrangères sont précédées de #.

3. **Contraintes d'intégrité référentielle :**

- NOTER.#num_élève → ÉLÈVE.num_élève
- NOTER.#code → MATIÈRE.code
- ENSEIGNER.#num_prof → PROFESSEUR.num_prof
- ENSEIGNER.#code → MATIÈRE.code

Remarque

L'association ENSEIGNER entre PROFESSEUR et MATIÈRE est n :m, donc elle donne lieu à une table intermédiaire ENSEIGNER. Sans attribut propre, sa clé primaire est simplement la concaténation des deux clés.

Correction — Exercice 3.7 : Dénormalisation

Soit le MLD : CLIENT(num_client, nom, adresse), COMMANDE(num_commande, date, #num_client), DETAIL(#num_commande, #ref_produit, quantite), PRODUIT(ref_produit, libelle, prix), LIVRAISON(num_livraison, date, #num_commande), FACTURE(num_facture, date, montant, #num_commande).

1. **Pourquoi dénormaliser ?** Le schéma est très éclaté : pour obtenir le détail d'une commande, il faut joindre 4 tables (COMMANDE, DETAIL, PRODUIT, CLIENT). Pour un bulletin de livraison avec facture, il faut joindre 5 à 6 tables. Ces jointures multiples pénalisent les performances en lecture. La dénormalisation vise à **améliorer les performances** en réduisant le nombre de jointures.
2. **Dénormalisation proposée :** On peut fusionner LIVRAISON et FACTURE dans COMMANDE, car chaque livraison et chaque facture correspond à exactement une commande :
COMMANDE(num_commande, date, #num_client, date_livraison, date_facture, montant_facture)
Cette fusion élimine 2 tables et simplifie les requêtes de consultation des commandes.
On peut aussi ajouter les informations du client directement dans COMMANDE (nom_client, adresse_client) pour éviter la jointure avec CLIENT lors de l'affichage des commandes, mais cela crée de la redondance.
3. **Risques de la dénormalisation :**
 - **Redondance** : les mêmes informations sont stockées à plusieurs endroits.
 - **Anomalies de mise à jour** : en cas de modification (ex. : changement d'adresse du client), il faut mettre à jour plusieurs enregistrements.
 - **Anomalies de suppression** : la suppression d'une commande peut entraîner la perte d'informations sur la livraison.
 - **Incohérence** : risque de données contradictoires si les mises à jour ne sont pas synchronisées.

Remarque

La dénormalisation est un compromis : on sacrifie la **qualité de la modélisation** (normalisation) au profit de la **performance en lecture**. Elle ne doit être envisagée que lorsque le schéma normalisé pose des problèmes de performances avérés, et toujours de manière raisonnée et documentée.

4 Chapitre 4 : Modélisation physique des données (MPD)**Correction — Exercice 4.1 : QCM — Du MLD au MPD**

1. **Réponse b)** — Le MPD est le MLD traduit dans le **langage d'un SGBD particulier**. Il spécifie les types de données, les index, les contraintes physiques, etc.
2. **Réponse d)** — Le passage du MLD au MPD nécessite de choisir les types de données, définir les index et créer les vues. Donc **toutes ces réponses**.
3. **Réponse b)** — La clause PRIMARY KEY définit la clé primaire en SQL.
4. **Réponse b)** — La contrainte ON DELETE CASCADE signifie que **les enregistrements enfants sont automatiquement supprimés** lorsque l'enregistrement parent est supprimé.

Les autres options de ON DELETE

- ON DELETE RESTRICT : interdit la suppression du parent s'il a des enfants.
- ON DELETE SET NULL : met la clé étrangère à NULL dans les enfants.
- ON DELETE SET DEFAULT : met la clé étrangère à sa valeur par défaut.

Correction — Exercice 4.2 : Types de données SQL

Table	Attribut	Type SQL
CLIENT	num_client	INTEGER
CLIENT	nom	VARCHAR(50)
CLIENT	date_naissance	DATE
PRODUIT	prix_unitaire	DECIMAL(10,2)
PRODUIT	description	TEXT
COMMANDE	date_commande	DATE
COMMANDE	est_livree	BOOLEAN
PRODUIT	code_postal	CHAR(5)

Justifications :

- INTEGER pour les identifiants numériques.
- VARCHAR(50) pour les noms (longueur variable, max 50 caractères).
- DATE pour les dates.
- DECIMAL(10,2) pour les prix (10 chiffres au total, 2 après la virgule).
- TEXT pour les descriptions longues.
- BOOLEAN pour les attributs vrai/faux.
- CHAR(5) pour les codes postaux (longueur fixe de 5 caractères).

Remarque

VARCHAR est utilisé pour les chaînes de longueur variable ; CHAR pour les chaînes de longueur fixe (codes, abréviations). DECIMAL est préféré à FLOAT pour les montants financiers car il évite les erreurs d'arrondi.

Correction — Exercice 4.3 : MPD — Bibliothèque

En reprenant le MLD de l'exercice 3.2 :

```
CREATE TABLE LIVRE (  
    ISBN VARCHAR(17) PRIMARY KEY,  
    titre VARCHAR(200) NOT NULL,  
    auteur VARCHAR(100) NOT NULL,  
    annee_pub INTEGER  
);  
  
CREATE TABLE ABONNE (  
    num_carte INTEGER PRIMARY KEY,  
    nom VARCHAR(50) NOT NULL,  
    prenom VARCHAR(50) NOT NULL,  
    adresse VARCHAR(200)  
);  
  
CREATE TABLE EMPRUNTER (  
    ISBN VARCHAR(17) NOT NULL,  
    num_carte INTEGER NOT NULL,  
    date_emprunt DATE NOT NULL,  
    date_retour_prevue DATE NOT NULL,  
    PRIMARY KEY (ISBN, num_carte),  
    FOREIGN KEY (ISBN) REFERENCES LIVRE(ISBN),  
    FOREIGN KEY (num_carte) REFERENCES ABONNE(num_carte),  
    CHECK (date_retour_prevue > date_emprunt)  
);
```

Remarque

La contrainte CHECK (date_retour_prevue > date_emprunt) garantit que la date de retour prévue est toujours postérieure à la date d'emprunt. C'est une **contrainte de domaine** qui enrichit le MPD par rapport au MLD.

Correction — Exercice 4.4 : MPD — Cabinet médical

En reprenant le MLD de l'exercice 3.3 :

```
CREATE TABLE MEDECIN (  
    num_ordre INTEGER PRIMARY KEY,  
    nom VARCHAR(50) NOT NULL,  
    specialite VARCHAR(50),  
    tarif DECIMAL(8,2) NOT NULL  
);  
  
CREATE TABLE PATIENT (  
    num_secu VARCHAR(15) PRIMARY KEY,  
    nom VARCHAR(50) NOT NULL,  
    prenom VARCHAR(50) NOT NULL,  
    date_naissance DATE NOT NULL  
);  
  
CREATE TABLE CONSULTER (  
    num_ordre INTEGER NOT NULL,  
    num_secu VARCHAR(15) NOT NULL,  
    date_consult DATE NOT NULL,  
    heure TIME NOT NULL,  
    diagnostic TEXT,  
    PRIMARY KEY (num_ordre, num_secu),  
    FOREIGN KEY (num_ordre) REFERENCES MEDECIN(num_ordre),  
    FOREIGN KEY (num_secu) REFERENCES PATIENT(num_secu)  
);  
  
CREATE TABLE ORDONNANCE (  
    num_ordonnance INTEGER PRIMARY KEY,  
    date_ord DATE NOT NULL,  
    num_secu VARCHAR(15) NOT NULL,  
    FOREIGN KEY (num_secu) REFERENCES PATIENT(num_secu)  
);  
  
CREATE TABLE MEDICAMENT (  
    code_medicament VARCHAR(10) PRIMARY KEY,  
    nom_medicament VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE PRESCRIRE (  
    num_ordonnance INTEGER NOT NULL,  
    code_medicament VARCHAR(10) NOT NULL,  
    posologie VARCHAR(200),  
    duree VARCHAR(50),  
    PRIMARY KEY (num_ordonnance, code_medicament),  
    FOREIGN KEY (num_ordonnance) REFERENCES ORDONNANCE(num_ordonnance),  
    FOREIGN KEY (code_medicament) REFERENCES MEDICAMENT(code_medicament)  
);
```

Remarque

L'ordre de création des tables est important en SQL : les tables référencées par des clés étrangères doivent être créées **avant** les tables qui les référencent. C'est pourquoi MEDECIN et PATIENT sont créées avant CONSULTER, et ORDONNANCE avant PRESCRIRE.

Correction — Exercice 4.5 : Index et optimisation

1. **Qu'est-ce qu'un index ?** Un index est une **structure de données** (souvent un arbre B+) associée à une table, qui permet d'accélérer la recherche des enregistrements sur une ou plusieurs colonnes. Il fonctionne comme un index dans un livre : au lieu de parcourir toutes les pages, on consulte l'index pour trouver directement la bonne page.
2. **Attributs pertinents pour un index :**
 - `ref_client` — très sollicité par les jointures (FOREIGN KEY).
 - `date_commande` — souvent utilisé dans les filtres par date.
 - `statut` — souvent filtré (ex. : `WHERE statut = 'en cours'`).La clé primaire `num_commande` possède déjà un index automatique (créé par PRIMARY KEY).

3. **Instructions SQL :**

```
CREATE INDEX idx_ref_client ON COMMANDE(ref_client);  
CREATE INDEX idx_date_commande ON COMMANDE(date_commande);  
CREATE INDEX idx_statut ON COMMANDE(statut);
```

4. **Inconvénients des index :**

- **Espace de stockage** : chaque index occupe de l'espace disque supplémentaire.
- **Ralentissement des écritures** : chaque INSERT, UPDATE ou DELETE doit mettre à jour les index en plus de la table.
- **Complexité** : trop d'index peut ralentir globalement les performances et compliquer la maintenance.

Remarque

Ne créez un index que sur des colonnes **fréquemment utilisées** dans les clauses WHERE, JOIN ou ORDER BY. Évitez d'indexer les colonnes rarement filtrées ou ayant très peu de valeurs distinctes (ex. : booléen).

Correction — Exercice 4.6 : Requêtes SQL sur le MPD

En utilisant le MPD de l'exercice 4.4 :

1. Patients nés après le 01/01/2000 :

```
SELECT * FROM PATIENT
WHERE date_naissance > '2000-01-01';
```

2. Nombre de consultations par médecin :

```
SELECT M.nom, COUNT(*) AS nb_consultations
FROM MEDECIN M
JOIN CONSULTER C ON M.num_ordre = C.num_ordre
GROUP BY M.num_ordre, M.nom;
```

3. Patients ayant eu plus de 3 consultations :

```
SELECT P.nom, P.prenom, COUNT(*) AS nb_consult
FROM PATIENT P
JOIN CONSULTER C ON P.num_secu = C.num_secu
GROUP BY P.num_secu, P.nom, P.prenom
HAVING COUNT(*) > 3;
```

4. Médecins cardiologues et leur nombre de consultations :

```
SELECT M.nom, COUNT(*) AS nb_consultations
FROM MEDECIN M
JOIN CONSULTER C ON M.num_ordre = C.num_ordre
WHERE M.specialite = 'Cardiologie'
GROUP BY M.num_ordre, M.nom;
```

5. Ordonnances prescrites entre le 01/01/2025 et le 30/06/2025 :

```
SELECT *
FROM ORDONNANCE
WHERE date_ord BETWEEN '2025-01-01' AND '2025-06-30';
```

Rappels SQL

- **GROUP BY** regroupe les lignes ayant les mêmes valeurs.
- **HAVING** filtre les groupes (comme **WHERE** filtre les lignes).
- **BETWEEN** est inclusif des deux bornes.
- **JOIN** effectue une jointure interne (seules les lignes correspondantes sont retournées).